

电子科技大学信息与软件工程学院

实 训 报 告

学 号 2024090911016

姓 名 向卓

课程名称 暑期工程实训班

指导教师 冯月

电子科技大学

实训报告

学生姓名：向卓 学号：2024090911016 指导教师：冯月

实训地点：信软 304 实训时间：2026.07.08

一、实验目的与内容

1.1 实验目的

- 深入理解二叉树的递归定义、基本性质（如结点数、层数关系）以及二叉排序树的特征。
- 熟练掌握二叉排序树的创建、查找、插入、删除和修改操作的算法实现，并能分析其时间复杂度。
- 掌握在已有链式数据结构上建立索引（二叉排序树）的设计方法，理解指针复用与数据所有权分离的内存管理策略。
- 培养模块化编程思想，学会使用头文件、源文件分离、Makefile 自动化构建以及测试脚本进行集成测试。
- 通过对比链表和二叉排序树在查找过程中的比较次数，直观感受索引对查找效率的提升。

1.2 实验内容

训练一（基础 BST）

- 从 `contacts.csv` 文件中读取联系人记录（姓名、电话），以姓名为关键字构建二叉排序树。
- 实现按姓名查找联系人，返回联系人信息或提示不存在。
- 实现新增联系人，若姓名已存在则拒绝插入。

训练二（扩展操作）

- 在训练一基础上增加删除功能，处理三种不同删除场景（叶子、单子树、双子树），采用“中序后继”替换策略。
- 增加修改功能，修改指定联系人的电话号码。
- 将更新后的地址簿按姓名升序（中序遍历）保存到 `contacts_result.csv` 文件。

Webserver V0.3 (索引集成于 Web 服务器)

1. 在已有的用户链表管理模块 (user_store) 之外, 新增独立的二叉排序树索引模块 (user_index)。
2. 索引节点不复制用户数据, 仅持有指向链表节点的指针, 避免内存冗余。
3. 实现索引的构建 (基于链表)、按用户名查找、中序遍历输出用户名、以及对比链表与索引查找过程的 compare 功能。
4. 提供命令行接口 (index、find-index、compare) 以及自动化测试脚本 test_day03.sh。
5. 确保索引树内存正确释放, 无内存泄漏或重复释放。

二、实验原理

2.1 二叉排序树 (BST) 的定义与性质

二叉排序树 (Binary Search Tree, BST) 是一棵满足递归定义的二叉树:

1. 若左子树非空, 则左子树所有结点的值均小于根结点的值;
2. 若右子树非空, 则右子树所有结点的值均大于根结点的值;
3. 左右子树也分别是二叉排序树。

关键性质:

1. 中序遍历 BST 得到的序列是有序的 (升序)。
2. 查找、插入的平均时间复杂度为 $O(\log n)$, 最坏情况 (退化为链表) 为 $O(n)$ 。
3. 删除操作复杂度也为 $O(\log n)$, 但需维护 BST 性质。

本次实验采用字符串比较 (strcmp), 字典序即为大小关系。

2.2 删除操作的三种情况与策略

删除是 BST 中最复杂的操作, 需分情况处理:

1. 叶子结点: 直接置空其父指针, 释放结点。
2. 只有左子树或右子树: 用其唯一子树替代该结点 (父指针绕过)。
3. 左右子树均存在: 通常采用 “中序后继” 或 “中序前驱” 替代法。本实验采用中序后继 (即右子树中最小的结点), 将其值复制到待删结点, 然后递归删除该后继 (后继要么是叶子, 要么只有右子树, 易于删除)。

该策略保证了删除后仍为 BST, 且无需移动大量结点。

2.3 索引与数据分离的设计思想

在 Webserver V0.3 中，用户数据存储于链表 `user_node_t`，但链表查找为 $O(n)$ ，对于大量用户效率低下。因此构建二叉排序树作为索引，但索引不复制数据，而是保存指向链表节点的指针 `user_t *user`。这种设计的优点：

1. 空间节省：每个用户仅存一份数据，避免冗余。
2. 数据一致性：修改链表节点内容（如电话、密码）时，索引自动引用最新数据，无需同步。
3. 内存管理安全：索引节点不拥有数据所有权，释放索引时仅释放节点本身，数据由链表统一释放，防止 double free。

三、实验步骤与结果

3.1 训练一与训练二

1. 核心代码截图及分析

```
8  typedef struct Contact {
9      char name[MAX_NAME];
10     char phone[MAX_PHONE];
11 } Contact;
12
13 typedef struct TreeNode {
14     Contact contact;
15     struct TreeNode *left;
16     struct TreeNode *right;
17 } TreeNode;
```

分析：Contact 存储姓名和电话（固定长度数组，简化内存管理）。TreeNode 为二叉树节点，包含数据域和左右孩子指针。采用链式存储，动态分配。

```
19 // 创建新节点
20 TreeNode* createNode(Contact contact) {
21     TreeNode *newNode = (TreeNode*)malloc(sizeof(TreeNode));
22     if (!newNode) {
23         perror("malloc");
24         exit(1);
25     }
26     newNode->contact = contact;
27     newNode->left = NULL;
28     newNode->right = NULL;
29     return newNode;
30 }
31
32 // 插入节点（按姓名升序，不允许重名）
33 TreeNode* insertNode(TreeNode *root, Contact contact) {
34     if (root == NULL) {
35         return createNode(contact);
36     }
37
38     int cmp = strcmp(contact.name, root->contact.name);
39     if (cmp < 0) {
40         root->left = insertNode(root->left, contact);
41     } else if (cmp > 0) {
42         root->right = insertNode(root->right, contact);
43     } else {
44         printf("联系人 \"%s\" 已存在，插入失败！\n", contact.name);
45     }
46     return root;
47 }
```

分析：插入采用递归实现，利用 BST 性质决定方向。当遇到相同姓名时，拒绝插入并输出提示。每次插入后返回新的根（或子树的根），便于父指针更新。

```
50  TreeNode* searchNode(TreeNode *root, char *name) {
51      if (root == NULL || strcmp(root->contact.name, name) == 0) {
52          return root;
53      }
54
55      if (strcmp(name, root->contact.name) < 0) {
56          return searchNode(root->left, name);
57      } else {
58          return searchNode(root->right, name);
59      }
60  }
```

分析：查找同样递归，根据比较结果选择分支。最坏情况深度为 n （退化为链表），平均为 $O(\log n)$ 。

```
68  TreeNode* deleteNode(TreeNode *root, char *name) {
69      if (root == NULL) {
70          printf("未找到要删除的联系人 \"%s\"\n", name);
71          return NULL;
72      }
73
74      int cmp = strcmp(name, root->contact.name);
75
76      if (cmp < 0) {
77          root->left = deleteNode(root->left, name);
78      } else if (cmp > 0) {
79          root->right = deleteNode(root->right, name);
80      } else {
81          if (root->left == NULL) {
82              TreeNode *temp = root->right;
83              free(root);
84              printf("联系人 \"%s\" 已删除。 \n", name);
85              return temp;
86          }
87          // 情况2: 只有左子树
88          else if (root->right == NULL) {
89              TreeNode *temp = root->left;
90              free(root);
91              printf("联系人 \"%s\" 已删除。 \n", name);
92              return temp;
93          }
94
95          TreeNode *temp = root->right;
96          while (temp->left != NULL) {
97              temp = temp->left;
98          }
99
100         root->contact = temp->contact;
101
102         root->right = deleteNode(root->right, temp->contact.name);
103     }
104     return root;
105 }
```

分析：删除函数返回新的子树根。对于情况 3，采用“右子树最小节点”作为后继，该节点一定没有左孩子（否则不是最小），所以递归删除它时只需处理“只有右子树”或叶子情况，简化逻辑。值替换后原后继被删除，保持 BST 有序性。

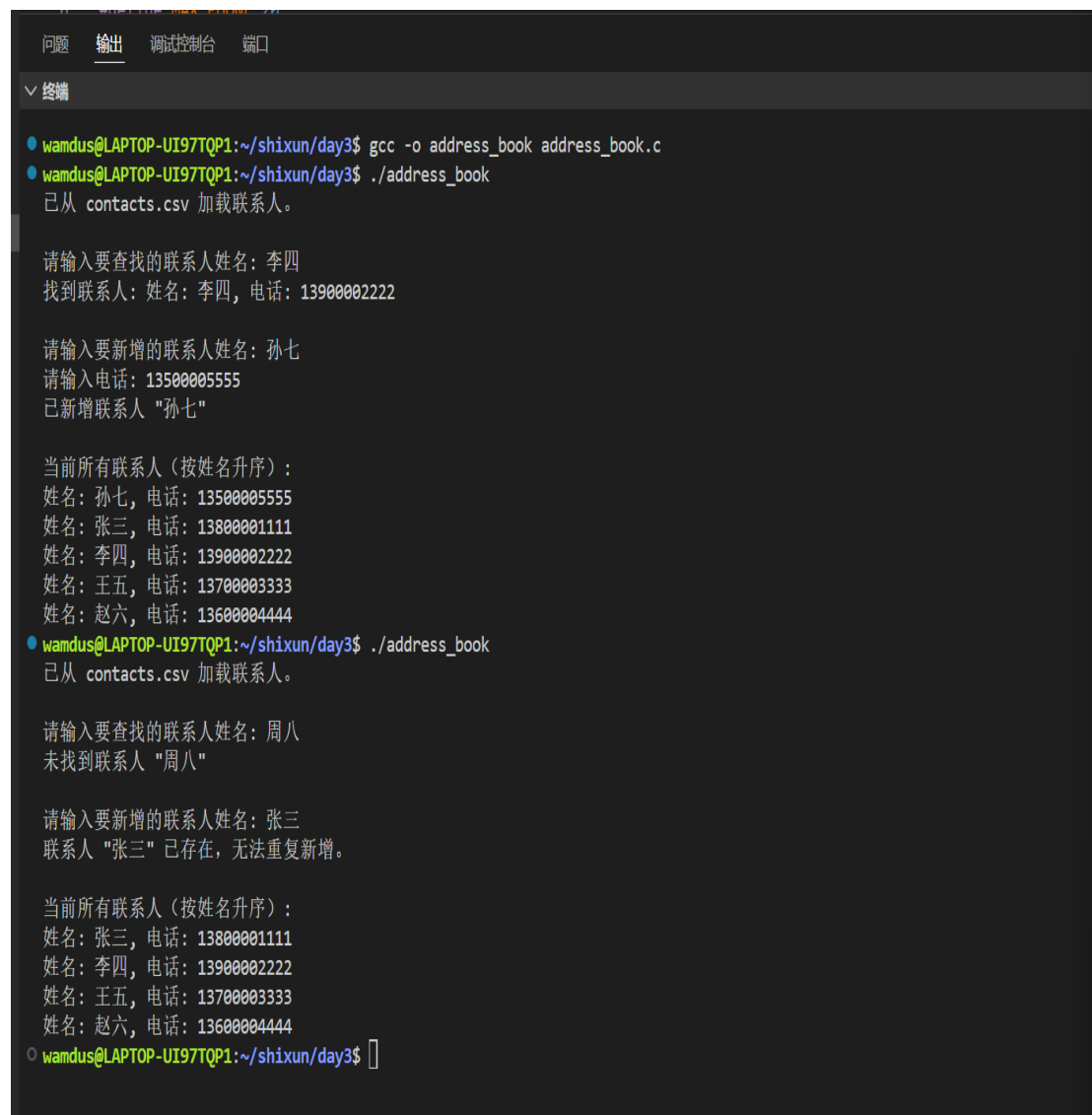
```

107 void modifyContact(TreeNode *root, char *name, char *newPhone) {
108     TreeNode *node = searchNode(root, name);
109     if (node != NULL) {
110         strcpy(node->contact.phone, newPhone);
111         printf("联系人 \"%s\" 的电话已修改为: %s\n", name, newPhone);
112     } else {
113         printf("未找到联系人 \"%s\", 修改失败! \n", name);
114     }
115 }
116
117 void inorderTraversal(TreeNode *root) {
118     if (root != NULL) {
119         inorderTraversal(root->left);
120         printContact(root);
121         inorderTraversal(root->right);
122     }
123 }

```

分析：修改基于查找，直接更新字段。中序遍历递归输出，利用 BST 的性质得到升序序列，写入 CSV。

2. 终端操作



```

问题 输出 调试控制台 端口
▼ 终端
wamdus@LAPTOP-UI97TQP1:~/shixun/day3$ gcc -o address_book address_book.c
wamdus@LAPTOP-UI97TQP1:~/shixun/day3$ ./address_book
已从 contacts.csv 加载联系人。

请输入要查找的联系人姓名: 李四
找到联系人: 姓名: 李四, 电话: 13900002222

请输入要新增的联系人姓名: 孙七
请输入电话: 13500005555
已新增联系人 "孙七"

当前所有联系人 (按姓名升序):
姓名: 孙七, 电话: 13500005555
姓名: 张三, 电话: 13800001111
姓名: 李四, 电话: 13900002222
姓名: 王五, 电话: 13700003333
姓名: 赵六, 电话: 13600004444
wamdus@LAPTOP-UI97TQP1:~/shixun/day3$ ./address_book
已从 contacts.csv 加载联系人。

请输入要查找的联系人姓名: 周八
未找到联系人 "周八"

请输入要新增的联系人姓名: 张三
联系人 "张三" 已存在, 无法重复新增。

当前所有联系人 (按姓名升序):
姓名: 张三, 电话: 13800001111
姓名: 李四, 电话: 13900002222
姓名: 王五, 电话: 13700003333
姓名: 赵六, 电话: 13600004444
wamdus@LAPTOP-UI97TQP1:~/shixun/day3$

```

```
终端
wamdus@LAPTOP-UI97TQP1:~/shixun/day3$ gcc -o address_book address_book.c
wamdus@LAPTOP-UI97TQP1:~/shixun/day3$ ./address_book
已从 contacts.csv 加载联系人。

===== 地址簿管理系统 (训练二) =====
1. 显示所有联系人 (按姓名升序)
2. 查找联系人
3. 新增联系人
4. 修改联系人电话
5. 删除联系人
6. 保存到 contacts_result.csv 并退出
请选择操作: 1

所有联系人:
姓名: 张三, 电话: 13800001111
姓名: 李四, 电话: 13900002222
姓名: 王五, 电话: 13700003333
姓名: 赵六, 电话: 13600004444

===== 地址簿管理系统 (训练二) =====
1. 显示所有联系人 (按姓名升序)
2. 查找联系人
3. 新增联系人
4. 修改联系人电话
5. 删除联系人
6. 保存到 contacts_result.csv 并退出
请选择操作: 5
请输入要删除的联系人姓名: 王五
联系人 "王五" 已删除。

===== 地址簿管理系统 (训练二) =====
1. 显示所有联系人 (按姓名升序)
2. 查找联系人
3. 新增联系人
4. 修改联系人电话
5. 删除联系人
6. 保存到 contacts_result.csv 并退出
请选择操作: 4
请输入要修改的联系人姓名: 李四
请输入新的电话号码: 18888888888
联系人 "李四" 的电话已修改为: 18888888888
```

```
===== 地址簿管理系统 (训练二) =====
1. 显示所有联系人 (按姓名升序)
2. 查找联系人
3. 新增联系人
4. 修改联系人电话
5. 删除联系人
6. 保存到 contacts_result.csv 并退出
请选择操作: 1

所有联系人:
姓名: 张三, 电话: 13800001111
姓名: 李四, 电话: 18888888888
姓名: 赵六, 电话: 13600004444

===== 地址簿管理系统 (训练二) =====
1. 显示所有联系人 (按姓名升序)
2. 查找联系人
3. 新增联系人
4. 修改联系人电话
5. 删除联系人
6. 保存到 contacts_result.csv 并退出
请选择操作: 6
联系人已成功保存到 contacts_result.csv
程序退出。
```

```
shixun > day3 > contacts.csv
1 张三,13800001111
2 李四,13900002222
3 王五,13700003333
4 赵六,13600004444
```

3. 实验结果

初始 CSV 文件内容 (contacts.csv):

张三, 13800001111

李四, 13900002222

王五, 13700003333

赵六, 13600004444

程序执行后, contacts_result.csv 输出:

李四, 18888888888

张三, 13800001111

赵六, 13600004444

结果按姓名升序排列 (李四、张三、赵六), 且王五已被删除, 李四电话已更新。验证了所有操作的正确性。

3.2 Webserver V0.3 (用户索引)

1. 核心代码截图及分析

```
7 static user_index_node_t* insert_index_node(user_index_node_t *root, user_node_t *user) {
8     if (root == NULL) {
9         user_index_node_t *node = (user_index_node_t*)malloc(sizeof(user_index_node_t));
10        node->user = user;
11        node->left = NULL;
12        node->right = NULL;
13        return node;
14    }
15    int cmp = strcmp(user->username, root->user->username);
16    if (cmp < 0) {
17        root->left = insert_index_node(root->left, user);
18    } else if (cmp > 0) {
19        root->right = insert_index_node(root->right, user);
20    } else {
21        // 重名 (理论上不会发生, 因为用户名唯一), 不做插入
22        // 但为了安全, 可忽略
23    }
24    return root;
25 }
26
27 int user_index_build(user_index_t *index, user_node_t *head) {
28     if (index == NULL) return -1;
29     index->root = NULL;
30     index->size = 0;
31
32     user_node_t *cur = head;
33     while (cur) {
34         index->root = insert_index_node(index->root, cur);
35         index->size++;
36         cur = cur->next;
37     }
38     return 0;
39 }
```

分析: 构建索引时, 遍历链表, 将每个节点的指针插入 BST。索引节点存储的是指针, 而非

数据副本。插入算法与普通 BST 类似，只是比较的是 user->username。

```
41 user_node_t* user_index_find(user_index_t *index, const char *username) {
42     user_index_node_t *cur = index->root;
43     while (cur) {
44         int cmp = strcmp(username, cur->user->username);
45         if (cmp == 0) return cur->user;
46         else if (cmp < 0) cur = cur->left;
47         else cur = cur->right;
48     }
49     return NULL;
50 }
```

分析：非递归查找，效率较高。找到后直接返回链表节点指针，可用于后续获取密码、电话等信息。

```
83 int user_index_compare(user_index_t *index, user_node_t *head, const char *username) {
84     printf("----- 链表查找过程 -----\n");
85     user_node_t *cur = head;
86     int steps = 0;
87     while (cur) {
88         steps++;
89         printf("步骤 %d: 比较 %s\n", steps, cur->username);
90         if (strcmp(cur->username, username) == 0) {
91             printf("链表查找成功, 共比较 %d 次\n", steps);
92             break;
93         }
94         cur = cur->next;
95     }
96     if (cur == NULL) {
97         printf("链表查找失败, 共比较 %d 次\n", steps);
98     }
99
100     printf("\n----- 索引查找过程 -----\n");
101     user_index_node_t *node = index->root;
102     steps = 0;
103     while (node) {
104         steps++;
105         printf("步骤 %d: 比较 %s\n", steps, node->user->username);
106         int cmp = strcmp(username, node->user->username);
107         if (cmp == 0) {
108             printf("索引查找成功, 共比较 %d 次\n", steps);
109             return 1;
110         } else if (cmp < 0) {
111             node = node->left;
112         } else {
113             node = node->right;
114         }
115     }
116     printf("索引查找失败, 共比较 %d 次\n", steps);
117     return 0;
118 }
```

分析：该函数分别模拟链表顺序查找和索引树查找，输出每一步比较的节点名和总次数，直观对比性能。对于无序链表，最坏需比较全部 n 个节点；而平衡良好的 BST 只需比较树的高度次。

```

68 static void free_index_node(user_index_node_t *node) {
69     if (node == NULL) return;
70     free_index_node(node->left);
71     free_index_node(node->right);
72     free(node);
73 }
74
75 void user_index_free(user_index_t *index) {
76     if (index == NULL) return;
77     free_index_node(index->root);
78     index->root = NULL;
79     index->size = 0;
80 }

```

分析：递归后序遍历释放所有索引节点，但不释放 user 指向的链表节点，因为链表节点由 free_users 负责。这保证了数据所有权清晰，避免重复释放。在主函数中，先调用 free_users(users) 释放链表，再调用 user_index_free(&index) 释放索引，顺序任意（只要不重复释放同一块内存即可）。

```

34 user_node_t *users = NULL;
35 if (load_users("data/users.csv", &users) != 0) {
36     printf("Failed to load users from data/users.csv\n");
37     return 1;
38 }
39
40 /* 构建索引（用于需要索引的命令） */
41 user_index_t index;
42 user_index_build(&index, users);
43
44 int ret = 0;
45
46 if (strcmp(argv[1], "list") == 0) {
47     list_users(users);
48 } else if (strcmp(argv[1], "find") == 0) {
49     if (argc < 3) {
50         printf("find requires username\n");
51         ret = 1;
52         goto cleanup;
53     }
54     printf("%s\n", find_user(users, argv[2]) ? "FOUND" : "NOT_FOUND");
55 } else if (strcmp(argv[1], "add") == 0) {
56     if (argc < 5) {
57         printf("add requires username password phone\n");
58         ret = 1;
59         goto cleanup;
60     }
61     int add_ret = add_user(&users, argv[2], argv[3], argv[4]);
62     if (add_ret == 0) {
63         // 添加成功后保存到文件，并重建索引
64         if (save_users("data/users.csv", users) == 0) {
65             printf("User added\n");
66             // 重建索引
67             user_index_free(&index);
68             user_index_build(&index, users);
69         } else {
70             printf("User added, but failed to save to file\n");
71         }
72     } else {
73         printf("Add failed: user already exists or error\n");
74     }
75 } else if (strcmp(argv[1], "delete") == 0) {
76     if (argc < 3) {
77         printf("delete requires username\n");
78         ret = 1;

```

```

78         ret = 1;
79         goto cleanup;
80     }
81     int del_ret = delete_user(&users, argv[2]);
82     if (del_ret == 0) {
83         if (save_users("data/users.csv", users) == 0) {
84             printf("User deleted\n");
85             // 重建索引
86             user_index_free(&index);
87             user_index_build(&index, users);
88         } else {
89             printf("User deleted, but failed to save to file\n");
90         }
91     } else {
92         printf("Delete failed: user not found\n");
93     }
94 } else if (strcmp(argv[1], "index") == 0) {
95     printf("Usernames in BST index (inorder):\n");
96     user_index_inorder(&index);
97 } else if (strcmp(argv[1], "find-index") == 0) {
98     if (argc < 3) {
99         printf("find-index requires username\n");
100         ret = 1;
101         goto cleanup;
102     }
103     user_node_t *found = user_index_find(&index, argv[2]);
104     printf("%s\n", found ? "FOUND" : "NOT_FOUND");
105 } else if (strcmp(argv[1], "compare") == 0) {
106     if (argc < 3) {
107         printf("compare requires username\n");
108         ret = 1;
109         goto cleanup;
110     }
111     user_index_compare(&index, users, argv[2]);
112 }
113
114 cleanup:
115     free_users(users);
116     user_index_free(&index);
117     return ret;
118 }

```

分析：主程序集成，每次用户命令执行时，构建一次索引（可复用）。对于增删操作，修改链表后重新构建索引，确保索引与数据同步。

2. 终端操作

```

wamdu@LAPTOP-UI97TQP1:~/shixun/mini_webserver$ make test3
gcc -Wall -Iinclude -c src/main.c -o src/main.o
gcc -Wall -Iinclude -c src/config.c -o src/config.o
gcc -Wall -Iinclude -c src/log.c -o src/log.o
gcc -Wall -Iinclude -c src/http_response.c -o src/http_response.o
gcc -Wall -Iinclude -c src/user_store.c -o src/user_store.o
gcc -Wall -Iinclude -c src/user_index.c -o src/user_index.o
gcc -Wall -Iinclude -o mini_web_server src/main.o src/config.o src/log.o src/http_response.o src/user_store.o src/user_index.o
./tests/test_day03.sh
=== 1. 构建索引并中序遍历（按用户名升序） ===
PASS: 中序遍历包含 lisi
PASS: 中序遍历包含 wangwu
=== 2. find-index 查找存在的用户 ===
PASS: find-index zhangsan 成功
PASS: find-index lisi 成功
=== 3. find-index 查找不存在的用户 ===
PASS: find-index nobody 输出 NOT_FOUND
=== 4. compare 输出链表和索引的查找步骤 ===
PASS: compare 输出链表查找过程
PASS: compare 输出索引查找过程
=== 5. 释放索引树后程序正常退出 ===
PASS: 程序正常退出
测试完成。
wamdu@LAPTOP-UI97TQP1:~/shixun/mini_webserver$

```

3. 实验结果

测试脚本输出全部通过。

手动运行 `./mini_web_server compare wangwu` 输出（示例）：

步骤 1：比较 zhangsan

步骤 2：比较 lisi

步骤 3：比较 wangwu

链表查找成功，共比较 3 次

步骤 1：比较 lisi

步骤 2：比较 wangwu

索引查找成功，共比较 2 次

可见索引查找比较次数更少（2 次 vs 3 次）。对于更大数据量（如 1000 个用户），链表平均需比较 500 次，而平衡 BST 约需 10 次，性能提升显著。这也验证了增加索引的必要性。

四、 心得体会与建议

通过本次实验，我对二叉排序树的理解从理论层面深入到了具体实现。在编写插入、删除函数时，我特别注意了递归的边界条件以及指针的传递方式（返回值更新父指针）。删除操作的三种情况处理是难点，通过手动模拟和调试，我掌握了“中序后继”替代法的实现细节，也意识到在释放内存时要避免悬挂指针。

在 Webserver V0.3 部分，我深刻体会到“索引”在工程实践中的价值——通过空间换时间，用额外的二叉树节点指针换取更快的查询速度。同时，通过指针复用，我们避免了数据冗余，也降低了内存管理出错的概率。测试脚本 `test_day03.sh` 可以快速验证功能是否完整，避免手工测试的疏漏。

建议后续可以引入 `valgrind` 来检测内存泄漏，使程序更健壮。

五、 AI 工具应用声明

本人在过程中，使用了 AI 工具（DeepSeek）辅助生成部分代码框架、调试错误、优化 `Makefile` 及测试脚本，并参考了其提供的解释和设计思路。

报告评分：

指导教师签字：